

An important feature is that when a project is recompiled after a few changes, it will recompile only the files which are changed, and any components that are dependent on it. This saves a lot of time. *Make* is, therefore, an essential tool for a large software project.

Each project needs a *Makefile*—a script that describes the project structure, namely, the source code files, the dependencies between them, compiler arguments, and how to produce the target output (normally, one or more executables). Whenever the *make* command is executed, the *Makefile* in the current working directory is interpreted, and the instructions executed to produce the target outputs. The *Makefile* contains a collection of rules, macros, variable assignments, etc. ('*Makefile*' or '*makefile*' are both acceptable.)

Installing GNU Make

Most distributions don't ship *make* as part of the default installation. You have to install it, either using the package-management system, or by manually compiling from source. To compile and build from source, download the tarball from <ftp://ftp.gnu.org/gnu/make/make-3.75.tar.gz>, extract it, and go through the README file. (If you're running Ubuntu, you can install *make* as well as some other common packages required for building from source, by running: `sudo apt-get install build-essential`.)

A sample project

To acquaint ourselves with the basics of *make*, let's use a simple C 'Hello world' project, and a *Makefile* that handles building of the target binary. We have three files (below): *module.h*, the header file that contains the declarations; *module.c*, which contains the definition of the function defined in *module.h*; and the main file, *main.c*, in which we call the *sample_func()* defined in *module.c*. Since *module.h* includes the required header files like *stdio.h*, we don't need to include *stdio.h* in every module; instead, we just include *module.h*. Here, *module.c* and *main.c* can be compiled as separate object modules, and can be linked by GCC to obtain the target binary.

module.h:

```
#include <stdio.h>
void sample_func();
```

module.c:

```
#include "module.h"
void sample_func()
{
    printf("Hello world!");
}
```

main.c:

```
#include "module.h"
void sample_func();
int main()
{
    sample_func();
    return 0;
}
```

The following are the manual steps to compile the project and produce the target binary:

```
slynux@freedom:~$ gcc -I . -c main.c # Obtain main.o
slynux@freedom:~$ gcc -I . -c module.c # Obtain module.o
slynux@freedom:~$ gcc main.o module.o -o target_bin #Obtain target binary
```

(-I is used to include the current directory (.) as a header file location.)

Writing a Makefile from scratch

By convention, all variable names used in a *Makefile* are in upper-case. A common variable assignment in a *Makefile* is *CC = gcc*, which can then be used later on as *\$(CC)* or *%\$(CC)*. *Makefiles* use '#' as the comment-start marker, just like in shell scripts.

The general syntax of a *Makefile* rule is as follows:

```
target: dependency1 dependency2 ...
[TAB] action1
[TAB] action2
...
```

Let's take a look at a simple *Makefile* for our sample project:

```
all: main.o module.o
    gcc main.o module.o -o target_bin

main.o: main.c module.h
    gcc -I . -c main.c

module.o: module.c module.h
    gcc -I . -c module.c

clean:
    rm -rf *.o
    rm target_bin
```

We have four targets in the *Makefile*:

- *all* is a special target that depends on *main.o* and *module.o*, and has the command (from the 'manual' steps earlier) to make GCC link the two object files into the final executable binary.
- *main.o* is a filename target that depends on *main.c* and *module.h*, and has the command to compile *main.c* to produce *main.o*.